

Where apecs stands

The whole measured surface as it is today: eight benchmarks against bitECS, koota and becsy with a hand-written typed-array loop as the floor, the nine SPEC §12.1 budgets, both known gaps, and what a query in key order costs.

machine Apple M1 node v20.19.0 suite 6 September 2026 comparison 6 September 2026 · min of 3 runs

0.97×

The hot loop, against a hand-written typed-array loop. 100 000 entities, two traits, one integrate pass through chunks.

38.1 B

Per entity, where the payload itself is 16 and the next-lightest library charges 154.

8 of 9

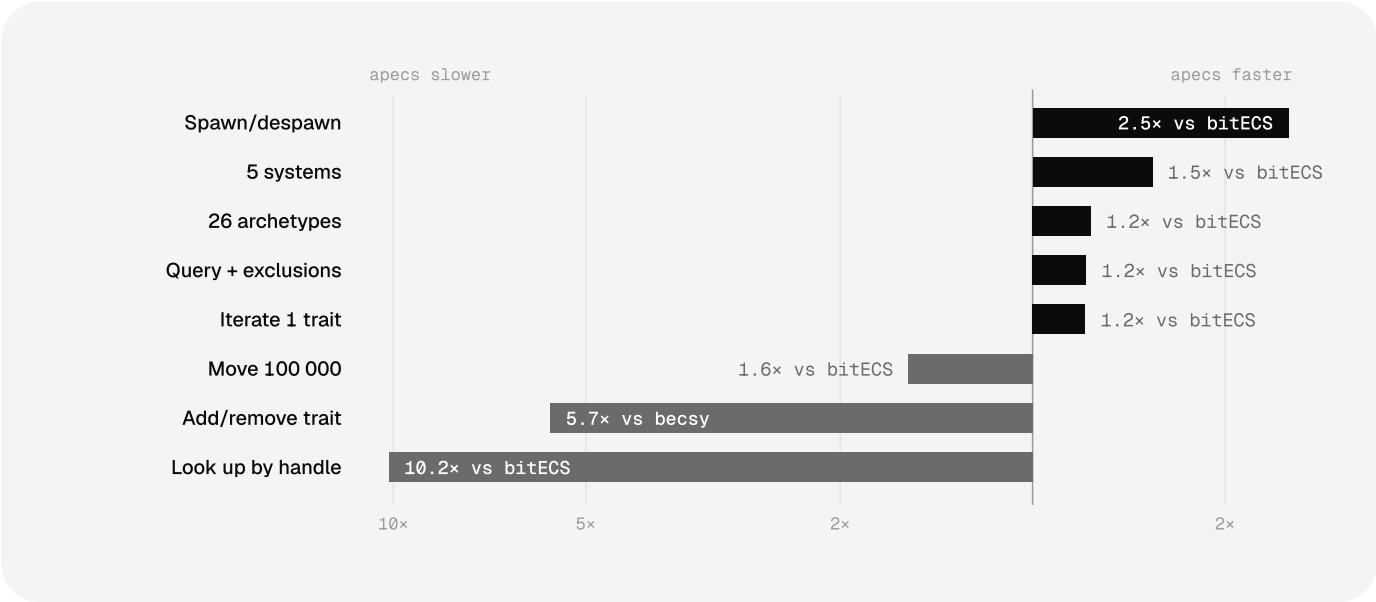
SPEC §12.1 budgets held. The miss is per-entity access by handle, at 37.45× of a flat array.

16.2×

The next ergonomic tier, at eight traits. apecs holds ~2.0 ns per entity visit at every trait count; koota and becsy lose an inline and never get it back.

Against the field

apecs against the fastest of bitECS, koota and becsy on each benchmark, log scale. Each library runs its own fastest correct idiom, one process per pair, and no timing is published until every library has matched the same entity count.



The losses are the same story: work that touches one entity at a time rather than a whole column. Iteration and query matching, where a column is the unit, are wins.

	hand-written	apecs	bitECS	koota	becsy
Iterate 1 trait	1.0×	1.2×	1.4×	2.9×	4.8×
5 systems	1.0×	0.9×	1.4×	2.3×	4.1×
Move 100 000	1.0×	2.1×	1.3×	4.2×	24.8×
26 archetypes	1.0×	0.9×	1.1×	3.4×	3.6×
Spawn/despawn	1.0×	58.7×	147×	301×	487×
Add/remove trait	1.0×	156×	132×	199×	27.5×
Query + exclusions	1.0×	0.8×	1.0×	3.0×	3.1×
Look up by handle	1.0×	12.6×	1.2×	2.0×	6.7×

lower is better · 1.0× is the hand-written typed-array loop

Every cell as a multiple of the hand-written floor; the fastest library in each row is the blue one. apecs beats the floor outright on three rows, which is what a column-shaped query buys over an index-shaped one.

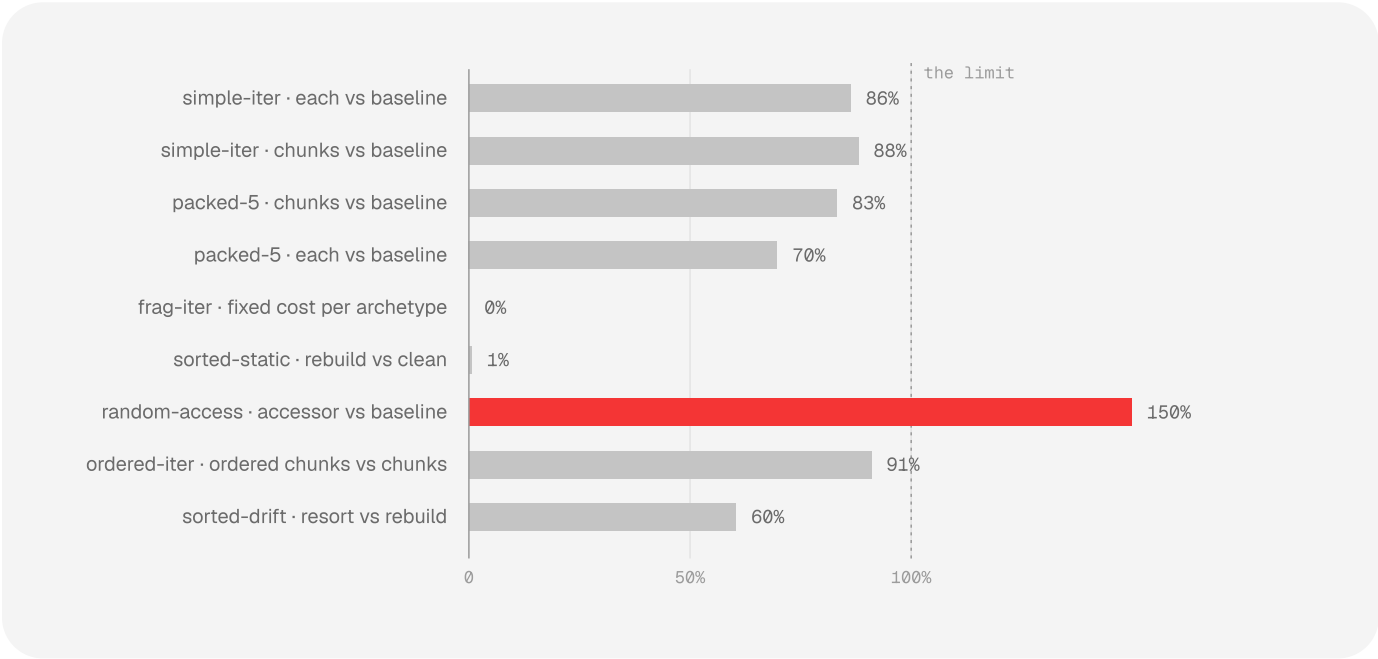
WHAT IT DOES	HAND-WRITTEN	APECS	BITECS	KOOTA	BECSY
Iterate one trait	4.0	4.7 1.2×	5.6 1.4×	11.5 2.9×	19.1 4.8×

WHAT IT DOES	HAND-WRITTEN	APECS	BITECS	KOOTA	BECSY
Five systems in a row	5.4	4.9 0.9×	7.6 1.4×	12.6 2.3×	22.3 4.1×
Move 100 000 entities	122	252 2.1×	161 1.3×	513 4.2×	3,021 24.8×
Iterate across 26 archetypes	105	94.1 0.9×	116 1.1×	355 3.4×	376 3.6×
Spawn and despawn	247	14,490 58.7×	36,413 147×	74,436 301×	120,303 487×
Add and remove a trait	132	20,572 156×	17,329 132×	26,158 199×	3,623 27.5×
Query with exclusions	60.4	47.4 0.8×	57.4 1.0×	180 3.0×	189 3.1×
Look up by entity handle	111	1,393 12.6×	137 1.2×	219 2.0×	737 6.7×

SPEC §12.1

Budgets

Each budget is a ratio between two benchmarks of the same run, never an absolute time: a laptop's microseconds say nothing, but “**chunks** is within 1.1× of the hand-written loop” holds anywhere. 8 of 9 hold; 1 does not.



Share of the budget spent, so all nine sit on one axis — the two budgets stated as “at least” are inverted to match. Anything past the dashed line is over.

BUDGET	MEASURED	LIMIT	\$12.1 TARGET	VERDICT
simple-iter · each vs baseline	2.16×	≤ 2.50×	1.5×	● within budget
simple-iter · chunks vs baseline	0.97×	≤ 1.10×	1.1×	● within budget
packed-5 · chunks vs baseline	1.08×	≤ 1.30×	parity	● within budget
packed-5 · each vs baseline	2.09×	≤ 3.00×	parity	● within budget
frag-iter · fixed cost per archetype	0ns	≤ 200ns	≤ 200ns	● within budget
sorted-static · rebuild vs clean	17098× cheaper	≥ 100× cheaper	zero work	● within budget
random-access · accessor vs baseline	37.45×	≤ 25.00×	25×	● over budget
ordered-iter · ordered chunks vs chunks	1.00×	≤ 1.10×	1.1×	● within budget
sorted-drift · resort vs rebuild	0.60×	≤ 1.00×	no full n log n	● within budget

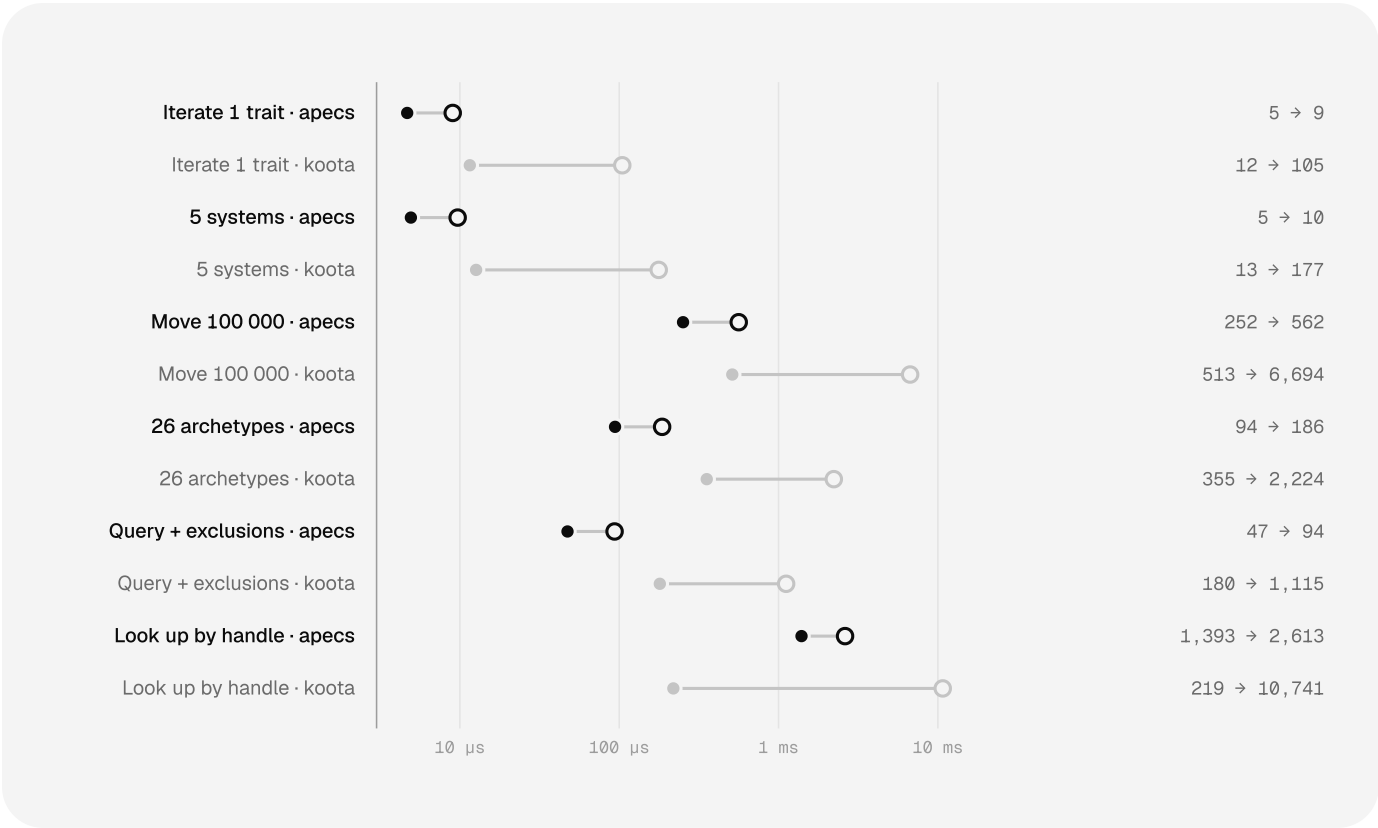
THE ONE MISS PREDATES THIS WORK

`world.accessor` resolves a field once and keeps a table per archetype, but three dependent loads against one flat array load is the archetype tax, and 25× was always the optimistic end of it. The iteration path is not involved and did not move.

What the escape hatch buys

The ergonomic idiom against the raw one, same benchmark and same data, log scale. bitECS has no ergonomic tier — the raw arrays are the API. becsy has no raw tier — everything goes through a system.

● raw tier ○ ergonomic tier

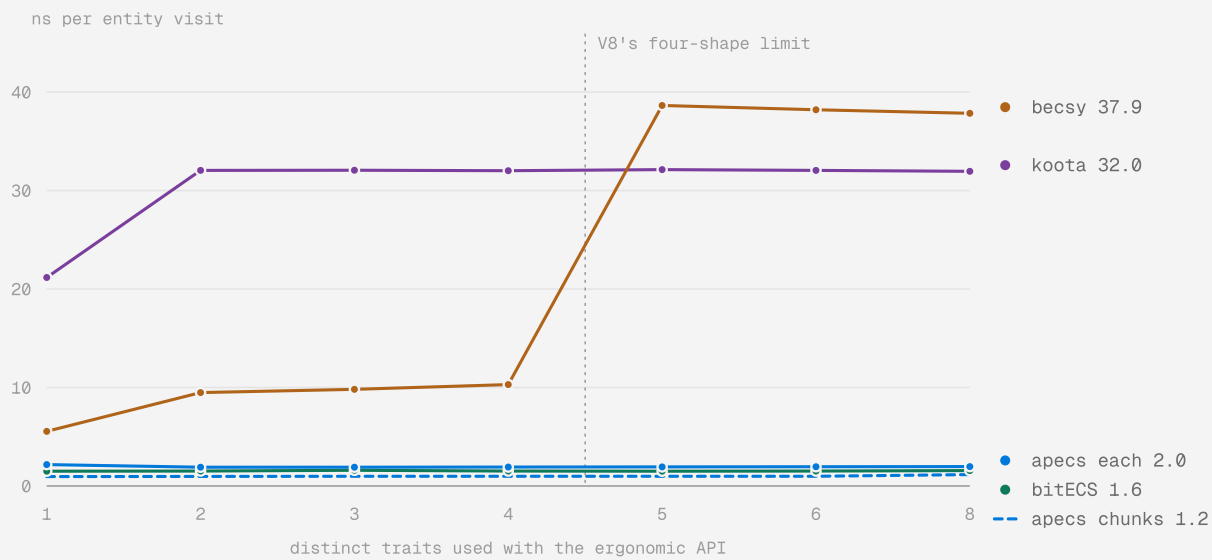


apecs pays about 2× to leave the raw arrays alone; koota pays between 6× and 58×. The numbers are microseconds — raw first, ergonomic second.

The five-trait cliff, and who pays it

Nanoseconds per entity visit against the number of distinct traits a program uses, one trait count per process — sweeping inside one process lets earlier traits pollute the dispatch site and manufactures a cliff whether or not it is there. Two of the four libraries charge for trait count, and they break in different places: becsy at the fifth trait, koota at the second.

● apecs each - - apecs chunks ● bitECS ● koota ● becsy

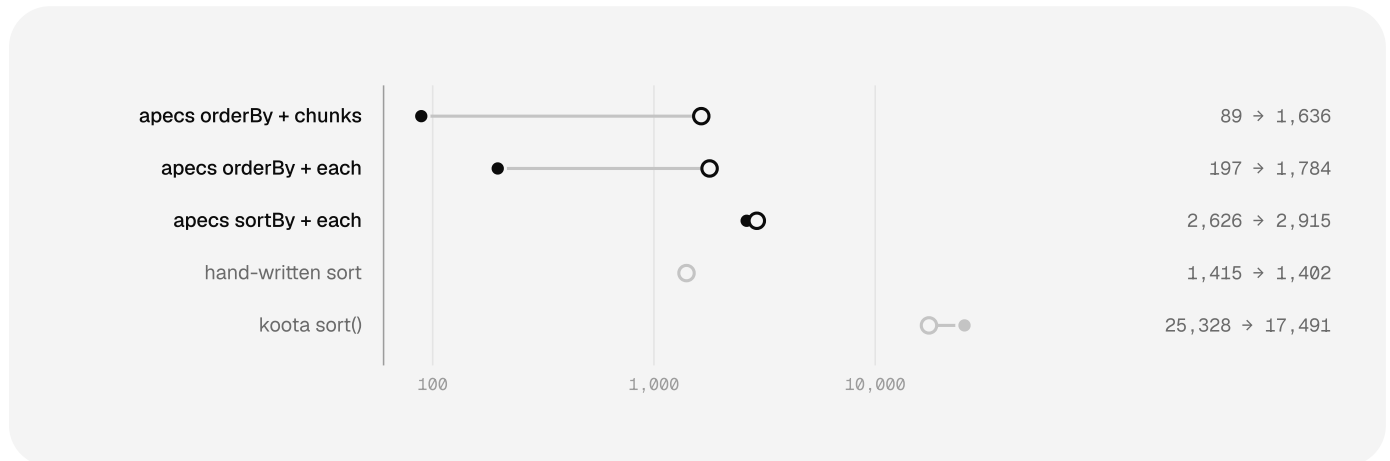


Nanoseconds per entity visit against the number of distinct traits a program uses with the ergonomic API. apecs is the fastest ergonomic tier at every trait count — bitECS's flat ~1.5 ns is its raw array API, not an ergonomic one. becsy steps at the fifth trait, koota at the second, and both are flat either side of the step.

Iterating in key order, every frame

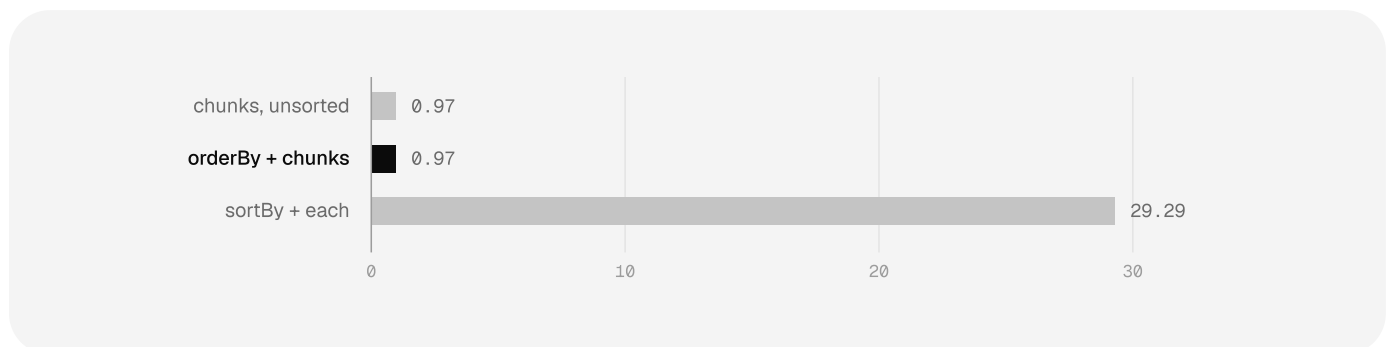
100 000 entities, microseconds per frame, log scale. bitECS and becsy ship no sorting, so their honest cost is the hand-written one: copy the ids out and sort them. Every apecs frame includes the `world.step()` a real frame pays — the dirty check is tick-based, and a view settles only once the tick has moved past the writes that built it.

● keys never change ○ 1% of keys change per frame



The distance between the two dots is what caching the order buys. koota's barely moves, because it re-sorts either way; apecs's `orderBy` is nearly two decades apart.

Sorted iteration is free; sorting is not. On a clean frame the rows are already in key order, so there is no gather and no side array to walk — 29.6× faster than apecs's own `sortBy`. When 1% of keys drift, `orderBy` extracts the keys, sorts, and applies the permutation to every column: 1.2× the hand-written sort, which moves one id array where apecs moves five columns of real data.



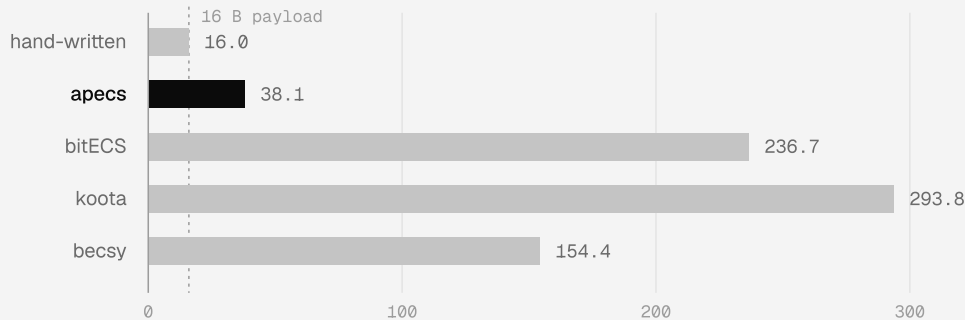
Nanoseconds per entity, all three from the same run. `sortBy` materialises the order into a side array and pays for the indirection on every row; `orderBy` leaves the rows in order and pays nothing.

THE GUARANTEE IS PER ARCHETYPE

Rows are in key order within each matching archetype, not across them — chunk boundaries are archetype boundaries, and a merge cannot cross one without materialising. `sortBy` remains the answer when the order must be total.

Bytes per entity

A world of entities carrying Position + Velocity — two `f32` fields each, so 16 bytes is the payload. Everything above it is ids, masks, archetype bookkeeping and query caches. Measured as absolute resident bytes at five world sizes and least-squares fitted; the check that the fit works is that the hand-written baseline lands on 16.



At a million entities: 38 MB for apecs against 237 MB for bitECS and 294 MB for koota.

READING THE FIT

It spans an empty world, so a fixed cost that moves between builds leaks into the slope. The marginal bytes between the two largest sizes cannot: 16.0 for hand-written, 36.9 for apecs, 239.8 for bitECS, 304.7 for koota, 153.4 for becsy.

METHOD

What would have made this wrong

One process per library, per benchmark

Four ECS libraries in one isolate make the shared iteration call sites megamorphic, and V8 deoptimises whichever warmed up second — a larger effect than most of the differences being measured.

A fairness census before any timing

Every library must match the same entity count on every query-shaped benchmark or the run aborts. It caught apecs's `Not()` silently ignoring all but its first argument, and becsy's `perf` build silently creating only some of the entities asked for.

Minimum of 3 full runs

One pass is not trustworthy on a laptop: across runs some competitor cells moved by tens of percent while apecs's moved under 2%. Worst spread observed here: `packed_1 · koota · raw` at 172%.

Production builds only

apecs from its built `dist/` with `__DEV__` compiled out; becsy from its `perf` build, whose default carries runtime validation.

Every measurement

The full suite behind the budgets above, as vitest reported it.

BENCHMARK	MEDIAN	MEAN	OPS/S	±RME
add-remove · apecs add/remove	41.5655 ms	42.4343 ms	24	2.9%
add-remove · apecs addMany/removeMany	39.7341 ms	41.0643 ms	24	5.8%
entity-cycle · baseline	0.2392 ms	0.2383 ms	4,196	0.1%
entity-cycle · apecs spawn/despawn	40.6979 ms	40.7483 ms	25	0.4%
frag-iter · apecs each x26	0.1857 ms	0.1877 ms	5,329	0.1%
frag-iter · apecs each x1	0.2118 ms	0.2129 ms	4,697	0.1%
packed-5 · baseline	0.0046 ms	0.0046 ms	215,063	0.0%
packed-5 · apecs each	0.0097 ms	0.0097 ms	102,588	0.2%
packed-5 · apecs chunks	0.0050 ms	0.0050 ms	198,022	0.1%
random-access · baseline	0.0842 ms	0.0846 ms	11,823	0.1%
random-access · apecs get/set	12.4407 ms	12.4721 ms	80	0.3%
random-access · apecs accessor	3.1520 ms	3.1589 ms	317	0.4%
schedule-dispatch · hand-written frame (16 systems)	0.0000 ms	0.0000 ms	28,339,632	0.2%
schedule-dispatch · schedule.run (16 systems)	0.0000 ms	0.0000 ms	26,345,802	0.5%
schedule-frame · hand-written frame (8 working systems)	0.2023 ms	0.2032 ms	4,922	0.1%
schedule-frame · schedule.run (8 working systems)	0.2047 ms	0.2061 ms	4,852	0.1%
simple-iter · baseline	0.2595 ms	0.2603 ms	3,841	0.1%
simple-iter · apecs each	0.5595 ms	0.5612 ms	1,782	0.1%
simple-iter · apecs chunks	0.2512 ms	0.2527 ms	3,958	0.1%
sorted-static · apecs sorted (clean)	0.0000 ms	0.0000 ms	27,172,006	0.0%
sorted-static · apecs sorted (rebuild)	0.7181 ms	0.7196 ms	1,390	0.1%

BENCHMARK	MEDIAN	MEAN	OPS/S	±RME
sorted-drift · apecs sorted (1% drift)	0.5092 ms	1.1373 ms	879	11.5%
sorted-drift · apecs drift writes	0.0757 ms	0.0760 ms	13,163	0.0%
ordered-iter · apecs ordered chunks	0.0966 ms	0.0970 ms	10,308	0.1%
ordered-iter · apecs sorted each	2.9294 ms	2.9734 ms	336	0.7%
ordered-iter · apecs chunks	0.0966 ms	0.0968 ms	10,334	0.1%

REPRODUCING

```
npm run bench:ci # SPEC §12.1 suite and budgets
cd bench/compare && node run.mjs # census, timings, memory
node capability.mjs apecs-ordered-chunks static
node ../../scripts/bench-report-html.mjs
```

apecs 0.1.0 (dist) · bitECS 0.4.0 · koota 0.6.6 · becsy 0.15.5 (perf build). Rendered from
bench/results.json and bench/compare/results.json; every figure on this page is read from a
results file, none is typed in.